



 Documentation Index

1. **Modal System** - modal1.md
2. **Form Validation** - valid1.md
3. **Theme Manager** - toggle_modes.md
4. **Dashboard Events** - both_homepage.md
5. **Public Events** - homepage_only.md
6. **Admin Dashboard** - admin_dashboard.md
7. **Student Dashboard** - student_dashboard.md
8. **History Handler** - history_handler.md

📌 **Note:** This document provides complete documentation for all 8 JavaScript files in the EventFinder project. Each file has been documented with functions, parameters, usage examples, and integration points.

1 modal1.js - Modal System

File Location: src/js/modal1.js | **Size:** ~250 lines | **Complexity:** Medium

Overview

The `modal1.js` file implements a reusable modal dialog system. It dynamically fetches HTML content from separate files, injects it into the DOM, handles script execution, and provides callbacks for initialization.

Core Functions

openModal(htmlPath, modalId, onReady)

Purpose: Fetches HTML, creates/updates modal, executes scripts, calls callback.

- **htmlPath:** Relative path to HTML file
- **modalId:** DOM ID for modal container
- **onReady:** Callback after modal is ready

Execution Flow:

```

openModal() called
|
| Check if container exists
|
| Fetch HTML from file
|
| Create/get modal container
|
| Parse .modal-content from response
|
| Set container innerHTML
|
| Execute inline scripts
|
| Call onReady callback
|
| Show modal with 'show' class

```

closeModal(modalId)

Purpose: Closes modal, removes 'show' class, clears content.

- Re-enables page scroll
- Removes modal from DOM
- Cleans up event listeners

Modal Structure

Expected HTML in loaded files:

```
<div class="modal-overlay">
  <div class="modal-dialog">
    <div class="modal-content">
      <!-- Form or content -->
    </div>
  </div>
</div>
```

Key Features

- **Dynamic Loading:** Loads HTML on demand from separate files
- **Script Execution:** Executes scripts within loaded HTML
- **Modal Caching:** Reuses modal element if already exists
- **Callbacks:** onReady parameter for post-load initialization
- **Error Handling:** Handles fetch failures gracefully

2 valid1.js - Form Validation

File Location: src/js/valid1.js | Size: ~300 lines | Complexity: Medium

Overview

The `valid1.js` file provides real-time form validation for signup and login pages. It validates fields, provides inline feedback, and prevents form submission with invalid data.

Validation Functions

validateFullName()

Regex: `^[A-Za-z\s]{2,50}$`

Accepts: 2-50 alphabetic characters and spaces. Rejects numbers and special characters.

validateEmail()

Regex: Standard email pattern

Uses helper functions: `isCollegeEmail()` and `isValidEmail()`

validatePassword()

Requirements:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (`@$!%*?&`)

Regex: `^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$$!%*?&]{8,}$`

validatePhone()

Format: Nepal phone numbers

Regex: `(98|97)\d{8}`

Accepts: 97 or 98 followed by 8 digits

College Email Validation

`isCollegeEmail(email)`

Regex: `[a-zA-Z]+\d+(bit|bca)\d+@kcc\.edu\.np`

Example valid format: `john123bit001@kcc.edu.np`

Form Handlers

handleLoginSubmit(e)

- Validates email and password
- Sends login request to PHP
- Handles authentication response
- Redirects on success

handleSignupSubmit(e)

- Validates all signup fields
- Checks password confirmation match
- Handles college email flow
- Creates user account

UI Feedback Functions

setFeedback(fieldId, message, isValid)

Shows inline validation message with color coding.

- Green: Valid input
- Red: Invalid input

setInputState(fieldId, isValid)

Updates input styling based on validation state.

updateSubmitButton()

Enables/disables submit button based on all field validity.

3 toggle_modes.js - Theme Manager

File Location: src/js/toggle_modes.js | Size: ~200 lines | Complexity: Low-Medium

Overview

The `toggle_modes.js` file implements a light/dark theme toggle system using a `ThemeManager` class. Theme preference is persisted in browser `localStorage`.

ThemeManager Class

Constructor: Creates theme manager instance and initializes.

Key Methods

`init()`

Initializes theme manager based on document readiness.

`setupEventListeners()`

Attaches click handlers to theme toggle buttons.

`toggle()`

Switches between light and dark modes.

```
const manager = new ThemeManager();
manager.toggle(); // Switches current theme
```

`setTheme(theme)`

Parameters: 'light' or 'dark'

Sets specific theme.

- Adds/removes `dark-mode` class on body
- Updates icon
- Saves to `localStorage`

`getTheme()`

Returns current theme: 'light' or 'dark'

CSS Integration

CSS handles color changes via `dark-mode` class:

```
body {
  background-color: white;
  color: #333;
}

body.dark-mode {
  background-color: #222;
  color: #fff;
}
```

Storage

`localStorage` Key: `'appTheme'`

Theme preference survives browser restart.

Accessibility

- Uses semantic HTML buttons
- ARIA labels for screen readers
- Keyboard accessible (tab to button, enter to toggle)

4

both_homepage.js - Dashboard Events

File Location: src/js/both_homepage.js | Size: ~150 lines | Complexity: Medium

Overview

The `both_homepage.js` file handles event loading and display for both admin and student dashboards. It manages event cards, details modals, and tab switching.

Core Functions

loadUpcomingEvents()

Fetches upcoming events and displays in grid layout.

- Endpoint: `get_upcoming_events.php`
- Container: `#eventsGrid`
- Shows event photo, name, category, date

loadRecentEvents()

Fetches recently completed events.

- Endpoint: `get_recent_events.php`
- Container: `#recentEventsGrid`

viewEventDetails(eventId)

Opens event details modal with full information.

- Loads `event_details_modal.html`
- Fetches event data from PHP
- Shows registration buttons
- Calculates event status (Upcoming/Completed/Today)

switchMainSection(sectionName)

Toggles between Home and Dashboard tabs.

- Parameters: `'home'` or `'dashboard'`
- Updates active tab styling
- Sets accessibility attributes

loadCurrentUserProfile()

Loads and displays user profile photo.

- Fetches from `get_current_user.php`
- Updates `#userProfilePic` src

Tab Structure

Tab	Container ID	Content
Home	<code>#homeSection</code>	Event grid
Dashboard	<code>#dashboardSection</code>	User-specific data

5 homepage_only.js - Public Events

File Location: src/js/homepage_only.js | **Size:** ~30 lines | **Complexity:** Low

Overview

The `homepage_only.js` file handles event loading for the public landing page (non-logged-in users). It shows upcoming and popular events but forces login to view details.

Core Functions

loadUpcomingEventsforOther()

Loads upcoming events for public visitors.

- **Endpoint:** ../php/get_upcoming_events.php
- **Container:** #eventsGrid
- Similar to dashboard version but redirects on click

loadPopularEvents()

Loads trending/popular events.

- **Endpoint:** ../php/get_popular_events.php
- **Container:** #popularEventsGrid
- Shows social proof (popular events attract sign-ups)

viewEventDetails(eventId)

Redirects non-logged-in users to login page.

Flow: Clicking "View Details" → Redirects to login.html → Prevents unauthorized access to event details

Initialization

Event: DOMContentLoaded

Both loading functions called when page fully loads.

6 admin_dashboard.js - Admin Dashboard

File Location: src/js/admin_dashboard.js | Size: ~200 lines | Complexity: Medium-High

Overview

The `admin_dashboard.js` file implements admin management interface for events, users, complaints, feedback, items, and requests using a generic modal system.

Module Variables

- `currentListType` - Track which list is displayed
- `currentListData` - Store fetched list data

Core Functions

openAdminListModal(type)

Opens generic list modal for different admin functions.

Supported Types:

- 'edit_event' - Events (editable)
- 'total_events' - All events (view-only)
- 'active_users' - Active users
- 'edit_users' - Users (editable)
- 'complaints' - Event complaints
- 'feedback' - Event feedback
- 'items_reported' - Lost/found items
- 'event_requests' - Student event requests

loadListData(type)

Fetches appropriate data based on list type.

- Maps type to PHP endpoint
- Fetches data
- Stores in `currentListData`
- Calls `renderAdminList()`

renderAdminList(data, type)

Renders appropriate HTML for each list type.

- Sets modal title
- Creates type-specific HTML
- Adds view/edit buttons

Endpoint Mapping

Type	Endpoint
edit_event / total_events	get_all_events.php
active_users / edit_users	get_all_users.php
items_reported	get_reported_items.php
event_requests	get_event_requests.php

7 student_dashboard.js - Student Dashboard

File Location: src/js/student_dashboard.js | Size: ~120 lines | Complexity: Medium

Overview

The `student_dashboard.js` file implements student management interface for personal activities (registrations, requests, complaints, feedback).

Core Functions

`openStudentListModal(type)`

Opens modal displaying different student lists.

Supported Types:

- 'my_registrations' - Events registered for
- 'my_event_requests' - Events requested
- 'my_complaints' - Complaints submitted
- 'my_feedback' - Feedback submitted
- 'participated' - Events participated in

`loadStudentListData(type)`

Fetches student-specific data based on list type.

`renderStudentList(data, type)`

Renders type-specific HTML with student-appropriate actions.

`viewEventDetailOnly(eventId)`

Opens event details modal (read-only for students).

List Types and Endpoints

Type	Endpoint
my_registrations	get_my_registrations.php
my_event_requests	get_my_event_requests.php
my_complaints	get_my_complaints.php
my_feedback	get_my_feedback.php
participated	get_participated_events.php

8 history_handler.js - History Handler

File Location: src/js/history_handler.js | Size: ~250 lines | Complexity: Medium (Pattern-based)

Overview

The `history_handler.js` file uses a configuration-based pattern to handle multiple history types (complaints, feedback, requests, items) without code duplication.

Configuration Object

`history_types`: Central configuration defining all history types.

History Types

Key	Purpose	Title
eventRequest	Student event requests	Event Request
complaint	Student complaints	Complaint
feedback	Event feedback/ratings	Feedback
reportedItem	Lost/found items	Reported Item

Core Functions

`openHistoryModal(type)`

Opens generic history modal for specified type.

`loadHistory(type)`

Fetches and displays history list for specified type.

`openHistoryDetail(type, id)`

Opens detail view for specific history item.

`searchHistory(type, searchTerm)`

Searches history items by term.

Design Pattern Benefits

- **DRY**: Single functions handle all types
- **Extensible**: Easy to add new types
- **Maintainable**: Centralized configuration
- **Consistent**: All types behave uniformly

 Summary

File Organization

The EventFinder JavaScript codebase is organized into 8 modular files, each handling specific functionality:

#	File	Purpose	Complexity
1	modal1.js	Reusable modal system	Medium
2	valid1.js	Form validation	Medium
3	toggle_modes.js	Theme switching	Low-Medium
4	both_homepage.js	Event loading	Medium
5	homepage_only.js	Public events	Low
6	admin_dashboard.js	Admin interface	Medium-High
7	student_dashboard.js	Student interface	Medium
8	history_handler.js	History management	Medium

Integration Map

Shared Dependencies:

- `modal1.js` - Used by all dashboard and history files
- `valid1.js` - Used by authentication pages
- `toggle_modes.js` - Used globally on all pages

Architecture Highlights

- Modular:** Each file handles specific domain
- Reusable:** Modal and validation systems used across files
- Pattern-based:** Configuration objects (`history_types`) reduce code duplication
- Frontend-heavy:** Minimal logic, focuses on UI interaction

⚠ Areas for Improvement:

- Code duplication in event rendering (`both_homepage.js`)
- Missing error handling on AJAX requests
- No pagination for large lists
- Hard-coded paths throughout files

EventFinder - JavaScript Documentation
Generated: May 22, 2026 | Version 1.0

This comprehensive documentation covers all 8 JavaScript files in the EventFinder project.